

API Key Management Location

Primary Location:

hypatiaux/config/api_keys.py

Alternative Location (More Secure):

hypatiaux/utils/api_key_manager.py

Complete API Key Management Script

api_key_manager.py - Secure API Key Management
Code

```
#!/usr/bin/env python3 """ API Key Manager for HypatiaX Secure storage, retrieval, and  
validation of API keys Location: hypatiaux/config/api_key_manager.py """ import os  
import json from pathlib import Path from typing import Optional, Dict from data
```

File Storage Locations

1. Main API Key Manager Script

hypatiaux/config/api_key_manager.py

OR

hypatiaux/utils/api_key_manager.py

2. .env File (Created by the manager)

hypatiaux/config/.env

3. .env.example (Template)

hypatiaux/.env.example

Directory Structure

hypatiax/

```
|— config/
|   |— __init__.py
|   |— base.py           #  Existing
|   |— api_key_manager.py #  NEW - Main script
|   |— .env              #  Created by manager (gitignored)
|   |— .api_keys.encrypted #  Optional encrypted storage
|   |— .encryption.key    #  Optional encryption key
|
|— .env.example           #  Template for users
|— .gitignore             #  Update to ignore secrets
```

Security Files to Ignore

Update your `.gitignore`:

bash

API Keys and Secrets

`.env`

`.api_keys.encrypted`

`.encryption.key`

`config/.env`

`config/.api_keys.encrypted`

`config/.encryption.key`



Create .env.example Template

Location: hypatiax/.env.example

bash

HypatiaX API Keys Configuration

Copy this file to .env and fill in your actual keys

Anthropic Claude (Recommended for 2025)

ANTHROPIC_API_KEY=your_anthropic_key_here

OpenAI GPT

OPENAI_API_KEY=your_openai_key_here

DeepSeek

DEEPSEEK_API_KEY=your_deepseek_key_here

HuggingFace (for models)

HUGGINGFACE_API_KEY=your_huggingface_token_here

Cohere (optional)

COHERE_API_KEY=your_cohere_key_here

Usage Examples

1. Interactive Setup (Recommended)

```
bash
```

```
cd hypatiaux
```

```
python config/api_key_manager.py setup
```

2. Set API Key Manually

```
bash
```

```
# Using system keyring (most secure)
```

```
python config/api_key_manager.py set ANTHROPIC --storage keyring
```

```
# Using .env file
```

```
python config/api_key_manager.py set OPENAI --storage env
```

3. Get API Key in Your Code

```
python
```

```
from config.api_key_manager import APIKeyManager
```

```
manager = APIKeyManager()
```

```
# Get Anthropic key
```

```
anthropic_key = manager.get_api_key("ANTHROPIC")
```

```
# Get OpenAI key
```

```
openai_key = manager.get_api_key("OPENAI")
```

If no key found, prompt user

```
if not anthropic_key:  
    anthropic_key = manager.prompt_for_api_key("ANTHROPIC",  
store=True)
```

4. List Configured Keys

bash

```
python config/api_key_manager.py list
```

5. Validate Keys

bash

```
python config/api_key_manager.py validate ANTHROPIC
```

6. Delete Keys

bash

```
python config/api_key_manager.py delete ANTHROPIC
```



Integration with Training Scripts

Update your training scripts to use the manager:

python

In hypatiax/core/training/training_llm.py

```
from config.api_key_manager import APIKeyManager
```

```
class ModernLLMTrainer:
```

```

def __init__(self, config: LLMConfig = None):
    self.config = config or LLMConfig()

    # Initialize API key manager
    self.key_manager = APIKeyManager()

    # Get API keys automatically
    if not self.config.anthropic_api_key:
        self.config.anthropic_api_key =
self.key_manager.get_api_key("ANTHROPIC")

    # Prompt if not found
    if not self.config.anthropic_api_key:
        print("⚠ Anthropic API key not found")
        self.config.anthropic_api_key =
self.key_manager.prompt_for_api_key(
    "ANTHROPIC",
    store=True
)
...

---

##  **Final Answer**

```

****Store the API key management script at:****

...

hypatiaux/config/api_key_manager.py

Install required dependencies:

bash

pip **install** keyring cryptography python-dotenv

Initialize:

bash

cd hypatiaux

python config/api_key_manager.py setup

This provides **3 storage methods** with automatic fallback:

1. **System Keyring** (most secure) 
2. **.env file** (convenient) 
3. **Encrypted file** (cross-platform) 

The script handles finding, creating, validating, and managing all API keys securely!



R

HUGGINGFACE_API_KEY--> No find out

You're right! Let me show you **exactly how to find and create your HuggingFace API key**:

How to Get Your HuggingFace API Key

Step 1: Go to HuggingFace Settings

<https://huggingface.co/settings/tokens>

Step 2: Sign In or Create Account

If you don't have an account:

- Go to: <https://huggingface.co/join>
- Sign up (free)

Step 3: Create New Token

1. **Navigate to:** <https://huggingface.co/settings/tokens>
2. **Click:** "New token" button
3. **Fill in:**
 - **Name:** HypatiaX (or any name you want)
 - **Type:** Choose one:
 - **Read** - For downloading models only
 - **Write** - For uploading models (if needed)
 - **Fine-grained** - Custom permissions
4. **Click:** "Generate a token"
5. **Copy** the token immediately (you won't see it again!)



Visual Guide

HuggingFace > Settings > Access Tokens	
[+ New token]	
Token name: HypatiaX	
Token type: ☒ Read ☐ Write ☐ Fine-grained	
[Generate a token]	

After generation:

Your new token:

hf_aBcDeFgHijKlMnOpQrStUvWxYz1234567890

⚠

Make sure to copy your token now.

You won't be able to see it again!

🔑 Token Format

HuggingFace tokens start with hf_ and look like:

hf_AbCdEfGhIjKlMnOpQrStUvWxYz1234567890AbCdEfGh

💾 Store Your Token

Option 1: Using the API Key Manager (Recommended)

```
bash
```

```
cd hypatix
```

```
python config/api_key_manager.py set HUGGINGFACE --storage keyring
```

Then paste your token when prompted.

Option 2: Manually in .env File

```
bash
```

Edit hypatiaux/config/.env

`nano config/.env`

Add this line:

`bash`

`HUGGINGFACE_API_KEY=hf_YourActualTokenHere`

Option 3: Environment Variable

`bash`

Linux/Mac

`export HUGGINGFACE_API_KEY=hf_YourActualTokenHere`

Windows (PowerShell)

`$env:HUGGINGFACE_API_KEY="hf_YourActualTokenHere"`

Windows (CMD)

`set HUGGINGFACE_API_KEY=hf_YourActualTokenHere`

Verify Your Token

Method 1: Using API Key Manager

`bash`

`cd hypatiaux`

`python config/api_key_manager.py validate HUGGINGFACE`

Method 2: Test in Python

python

```
from huggingface_hub import HfApi

api = HfApi()
user_info = api.whoami(token="hf_YourTokenHere")
print(f"✅ Logged in as: {user_info['name']}")
```

Method 3: Using CLI

bash

Install CLI

```
pip install huggingface_hub
```

Login

```
huggingface-cli login
```

Test

```
huggingface-cli whoami
```



Updated API Key Manager

I need to update the validator for HuggingFace. Here's the fix:

api_key_manager.py - Secure API Key Management

Code · Version 2

```
def validate_api_key(self, provider: str, api_key: str = None) -> bool: """ Validate API
key by making a test request Args: provider: Provider name api_key: API key to validate
(if None, us
```

Install Required Package

bash

pip **install** huggingface_hub

Quick Setup Guide

Complete Setup in 3 Steps:

bash

1. Get your token from HuggingFace

Visit: <https://huggingface.co/settings/tokens>

2. Store it using the API Key Manager

cd hypatiaux

python config/api_key_manager.py **set** HUGGINGFACE --storage keyring

3. Verify it works

python config/api_key_manager.py validate HUGGINGFACE

Direct Links

Action	URL
Create Account	https://huggingface.co/join
Create Token	https://huggingface.co/settings/tokens
Token Docs	https://huggingface.co/docs/hub/security-

Action

URL

[tokens](#)

Profile Settings

<https://huggingface.co/settings/profile>



What You'll Use It For

HuggingFace tokens are needed for:

1. Downloading Models:

python

```
from transformers import AutoModel

# Some models require authentication
model = AutoModel.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    use_auth_token=True # Uses your stored token
)
```

2. Uploading Models:

python

```
model.push_to_hub("username/model-name", use_auth_token=True)
```

3. Accessing Private Models:

python

```
# Access your private models or gated models
model = AutoModel.from_pretrained(
    "username/private-model",
    use_auth_token=your_token
```

)

✅ Summary

To get your HuggingFace API key:

1. Go to: <https://huggingface.co/settings/tokens>
2. Click "New token"
3. Name it "HypatiaX"
4. Select "Read" type
5. Click "Generate a token"
6. **Copy it immediately!**
7. Store it:

bash

```
python config/api_key_manager.py set HUGGINGFACE --storage keyring
```

The token will look like: hf_aBcDeFgHiJkLmNoPqRSTuVwXyZ1234567890

Done! 🎉

R

How add key from gmail account to google authenticator

I think there might be some confusion here. Let me clarify what you're asking about and provide the correct guidance:

🔑 Google Authenticator vs API Keys

What You Probably Mean:

1. **Google Authenticator (2FA)** - For securing your accounts
2. **API Keys** - For programmatic access to services

These are **different things**. Let me explain both:

Option 1: Enable 2FA on Gmail with Google Authenticator

If you want to **secure your Gmail account** with Google Authenticator:

Steps:

1. Go to Google Account Security:

<https://myaccount.google.com/security>

2. Enable 2-Step Verification:

- Click "2-Step Verification"
- Click "Get Started"
- Follow prompts to verify your phone

3. Add Google Authenticator:

- Scroll to "Authenticator app"
- Click "Set up"
- Choose your device (Android/iPhone)
- **Scan the QR code** with Google Authenticator app
- Enter the 6-digit code to verify

4. Save Backup Codes:

- Write down the backup codes
 - Store them safely
-

Option 2: Get Google Cloud API Keys

If you want **API keys for Google services** (like Google Cloud, Gmail API):

For Google Cloud Platform:

1. Go to Google Cloud Console:

<https://console.cloud.google.com/>

2. Create a Project:

- Click "Select a Project" → "New Project"
- Name it "HypatiaX"
- Click "Create"

3. Enable APIs:

- Go to "APIs & Services" → "Library"
- Search for the API you need (e.g., "Gmail API")
- Click "Enable"

4. **Create Credentials:**

- Go to "APIs & Services" → "Credentials"
- Click "Create Credentials" → "API Key"
- **Copy the API key immediately!**

5. **Restrict Your API Key (Important!):**

- Click on the API key name
- Under "API restrictions" → Select specific APIs
- Under "Application restrictions" → Choose appropriate restriction

For Gmail API (OAuth 2.0):

Gmail API uses OAuth 2.0, not simple API keys:

1. **Create OAuth Client:**

<https://console.cloud.google.com/apis/credentials>

- Click "Create Credentials" → "OAuth client ID"
- Choose "Desktop app" or "Web application"
- Download the JSON file

2. **Save the Credentials:**

json

```
{
  "client_id": "your-client-id.apps.googleusercontent.com",
  "client_secret": "your-client-secret",
  "redirect_uris": ["http://localhost"]
}
```

Step-by-Step: Google Authenticator Setup

On Your Phone:

1. Download Google Authenticator:

- **Android:** <https://play.google.com/store/apps/details?id=com.google.android.apps.authenticator2>
- **iPhone:** <https://apps.apple.com/app/google-authenticator/id388497605>

2. Open the App:

- Tap "Get started"
- Tap "Scan a QR code" or "Enter a setup key"

3. From Your Computer:

- Go to: <https://myaccount.google.com/security>
- Click "2-Step Verification"
- Scroll to "Authenticator app"
- Click "Set up"
- **Show the QR code on screen**

4. Scan with Phone:

- Point your phone camera at the QR code
- Google Authenticator will add your account
- You'll see a 6-digit code that changes every 30 seconds

5. Verify:

- Enter the 6-digit code from your phone
- Click "Verify"
- Done! 

Store Google Credentials in HypatiaX

If you want to store **Google API credentials** in your HypatiaX project:

google_credentials_manager.py - Google API Credentials

Code

```
#!/usr/bin/env python3 """ Google Credentials Manager for HypatiaX Handles Google
Cloud API keys and OAuth credentials Location:
hypatiaux/config/google_credentials_manager.py """ import os import json from pathlib
import Path from typing import Opti
```

Where to Store This

hypatiaux/config/google_credentials_manager.py

Usage

1. Setup Interactively:

bash

cd hypatiaux

python config/google_credentials_manager.py setup

2. Set API Key:

bash

python config/google_credentials_manager.py set-api-key --service gmail

3. Load OAuth Credentials:

bash

python config/google_credentials_manager.py load-oauth

~/Downloads/credentials.json

What You Actually Need

Based on your question, you probably want **ONE of these**:

Scenario 1: Secure Your Gmail Account

→ Use Google Authenticator (2FA) → Go to: <https://myaccount.google.com/security>

Scenario 2: Use Gmail API in HypatiaX

→ Get OAuth 2.0 credentials → Go to:

<https://console.cloud.google.com/apis/credentials>

Scenario 3: Use Google Cloud Services

→ Get API keys → Go to: <https://console.cloud.google.com/apis/credentials>

? Which One Do You Need?

Please clarify:

1. **Do you want to secure your Gmail account with 2FA?** (Google Authenticator)
2. **Do you want to use Gmail API in your HypatiaX code?** (OAuth 2.0)
3. **Do you want to use Google Cloud services?** (API Keys)

Let me know and I'll give you specific instructions! 🎯

R

Add cohere to api_key_management.py

I'll add Cohere to the API key manager script:

api_key_manager.py - Secure API Key Management

Code · Version 4

```
# Supported providers PROVIDERS = { "ANTHROPIC": "ANTHROPIC_API_KEY",  
"OPENAI": "OPENAI_API_KEY", "DEEPSEEK": "DEEPSEEK_API_KEY",  
"HUGGINGFACE": "HUGGINGFACE_API_KEY", "COHERE":  
"COHERE_API_KEY",
```

Perfect! Now let me create a comprehensive guide for getting your Cohere API key:

🔑 How to Get Your Cohere API Key

Step 1: Create Cohere Account

1. **Go to Cohere:**

<https://cohere.com/>

2. Sign Up:

- Click "Get Started" or "Sign Up"
- Use email or Google/GitHub account
- **Free tier available!** (1000 API calls/month)

Step 2: Access Dashboard

1. Go to Dashboard:

<https://dashboard.cohere.com/>

2. Navigate to API Keys:

<https://dashboard.cohere.com/api-keys>

Step 3: Generate API Key

1. Create New Key:

- Click "Create API Key" or "Generate API Key"
- Give it a name: `HypatiaX`
- Click "Create"

2. Copy Your Key:

- Format: `abc123XYZ . . .` (long string)
- **Copy it immediately!**
- Store it securely

Store Cohere Key in HypatiaX

Option 1: Using API Key Manager (Recommended)

`bash`

`cd hypatiaux`

Interactive setup

`python config/api_key_manager.py setup`

Then select "4. Cohere"

Or directly set

```
python config/api_key_manager.py set COHERE --storage keyring
```

Option 2: Manually in .env File

Edit `hypatiax/config/.env`:

bash

Add this line

```
COHERE_API_KEY=your_cohere_key_here
```

Option 3: Environment Variable

bash

Linux/Mac

```
export COHERE_API_KEY=your_cohere_key_here
```

Windows (PowerShell)

```
$env:COHERE_API_KEY="your_cohere_key_here"
```

Verify Your Cohere Key

Method 1: Using API Key Manager

bash

```
cd hypatiax
```

```
python config/api_key_manager.py validate COHERE
```

Method 2: Test in Python

python

```
import cohere
```

```
co = cohere.Client('your_cohere_key_here')
```

```
# Test generation
```

```
response = co.generate(  
    prompt="Hello, how are you?",  
    max_tokens=50  
)
```

```
print(response.generations[0].text)
```



Install Cohere SDK

bash

```
pip install cohere
```



Use Cohere in Your Training Scripts

python

```
# In hypatiax/core/training/training_llm.py
```

```
from config.api_key_manager import APIKeyManager
```

```
import cohere
```

```
class ModernLLMTrainer:
```

```
    def __init__(self, config: LLMConfig = None):
```

```
        self.config = config or LLMConfig()
```

```
        # Get Cohere key
```

```
        key_manager = APIKeyManager()
```

```
        cohere_key = key_manager.get_api_key("COHERE")
```

```
        if cohere_key:
```

```
            self.cohere_client = cohere.Client(cohere_key)
```

```
            print("✅ Cohere client initialized")
```

```
        else:
```

```
            print("⚠️ Cohere API key not found")
```

```
    def generate_with_cohere(self, prompt: str) -> str:
```

```
        """Generate using Cohere"""
```

```
        if not hasattr(self, 'cohere_client'):
```

```
            return "Error: Cohere not configured"
```

```
        try:
```

```
            response = self.cohere_client.generate(
```

```
                prompt=prompt,
```

```

        max_tokens=256,
        temperature=0.1,
        model="command" # or "command-light", "command-
nightly"
    )

    return response.generations[0].text
except Exception as e:
    return f"Error: {e}"

```

Cohere Models Available

Model	Best For	Cost
command	General purpose	Free tier: 1000 calls/month
command-light	Faster, lighter	More calls allowed
command-nightly	Latest features	Same as command
embed-english-v3.0	Embeddings	Text embeddings

Compare All Providers

After adding Cohere, here's your complete provider list:

```
bash
```

```
cd hypatiax
```

```
python config/api_key_manager.py list
```

```
...
```


Output:

...

 Configured API Keys:

=====

ANTHROPIC:  Configured

OPENAI:  Configured

DEEPSEEK:  Not configured

HUGGINGFACE:  Configured

COHERE:  Configured

GOOGLE:  Not configured

Cohere Links

Resource	URL
Sign Up	https://cohere.com/
Dashboard	https://dashboard.cohere.com/
API Keys	https://dashboard.cohere.com/ api-keys
Documentation	https://docs.cohere.com/
Pricing	https://cohere.com/pricing

Updated .env.example

Update your hypatiax/.env.example:

bash

HypatiaX API Keys Configuration

Anthropic Claude (Recommended for 2025)

ANTHROPIC_API_KEY=your_anthropic_key_here

OpenAI GPT

OPENAI_API_KEY=your_openai_key_here

DeepSeek

DEEPSEEK_API_KEY=your_deepseek_key_here

HuggingFace (for models)

HUGGINGFACE_API_KEY=your_huggingface_token_here

Cohere (for embeddings and generation)

COHERE_API_KEY=your_cohere_key_here

Google Gemini

GOOGLE_API_KEY=your_google_key_here



Complete Setup Commands

bash

cd hypatiaux

1. Run interactive setup

python config/api_key_manager.py setup

2. Select option 4 (Cohere)

Enter your Cohere API key when prompted

3. Verify it works

```
python config/api_key_manager.py validate COHERE
```

4. List all configured providers

```
python config/api_key_manager.py list
```



Summary



Cohere is now added to your API Key Manager!

To set up:

1. Get key from: <https://dashboard.cohere.com/api-keys>
2. Run: `python config/api_key_manager.py setup`
3. Select "4. Cohere"
4. Paste your key
5. Done!

The updated `api_key_manager.py` now supports **6 providers**:

1. Anthropic Claude
2. OpenAI GPT
3. DeepSeek
4. HuggingFace
5. **Cohere** ✨ (NEW!)
6. Google Gemini

All with validation, secure storage, and easy management! 